ENGDES26 — ENGINEERING DESIGN

Getting Started

ESP32-S3 starter firmware project — setup, build, and first flash

Generated October 10, 2026

EngDes26 — Getting Started

A step-by-step guide for getting this project built and running on your board, written assuming you've never used an ESP32 or ESP-IDF before. If you get stuck, the **Troubleshooting** section at the bottom covers the most common snags.

What you're working with

- **Board:** ESP32-S3 (the chip doing the work — I2C/SPI sensors, a 3-phase PWM output, an SD card slot, a status LED).
- **ESP-IDF:** Espressif's official development framework for this chip — the compiler toolchain, libraries, and the `idf.py` command-line tool you'll use to build and flash.

This starter project has WiFi removed (explained in `main/main.c`) and an optional OLED display support that's off by default (also in `main/main.c`, look for `OLED_ENABLED`).

1. Install VS Code and the ESP-IDF extension

1. Install **VS Code** from code.visualstudio.com if you don't have it.
2. Open the **Extensions** panel (`Ctrl+Shift+X`), search for **"Espressif IDF"**, and install it (publisher: Espressif Systems).

2. Run the setup wizard

1. Open the Command Palette (`Ctrl+Shift+P`) and run **"ESP-IDF: Configure ESP-IDF Extension"** (this may also pop up automatically the first time the extension activates).
2. Choose **"Express"** setup.

3. For **ESP-IDF version**, pick **v6.0**. Picking an older version will cause build errors (`component not found`), since this project uses newer ESP-IDF driver APIs.
4. Accept the suggested install location (usually something like `C:\Espressif` or a folder under your user profile). **Don't** redirect it into a folder that's synced by OneDrive or deeply nested — long/syncing paths occasionally break the toolchain install.
5. Click through and let it download — this pulls down the compiler toolchain, Python environment, and tools (a few GB), and can take 10-20 minutes depending on your connection. You only do this once per PC.

3. Open the project

1. **File** → **Open Folder…** and select this project's folder (the one containing `CMakeLists.txt`, `main/`, `components/`).
2. Open a terminal set up for ESP-IDF: Command Palette → **"ESP-IDF: Open ESP-IDF Terminal"**. (A plain VS Code terminal won't have `idf.py` on its PATH — always use this command, or the extension's sidebar buttons, so the right environment is active.)

4. First build

In that ESP-IDF terminal:

```
idf.py set-target esp32s3
idf.py build
```

`set-target` only needs to run once — it generates your own `sdkconfig` file from `sdkconfig.defaults`. `build` compiles the project; the first build takes a few minutes, later builds are much faster.

5. First flash: connect the board and enter bootloader mode

Flashing means writing your compiled program into the board's flash memory. To do that the very first time, the chip has to be manually put into **bootloader mode** — later flashes won't need this (see the note at the end of this section).

1. Plug the board into your PC with a **USB-C cable** (data-capable, not a charge-only cable).
2. **Hold down** the **BOOT** button.
3. While still holding BOOT, **press and release** the **RESET** button.
4. **Release** the BOOT button.

The board is now sitting in bootloader mode, waiting to be flashed.

Find the port

Device Manager → "Ports (COM & LPT)" → look for something like `COM5`. (The ESP-IDF extension can also do this for you — Command Palette → **"ESP-IDF: Select Port to Use"**.)

Flash and monitor

```
idf.py -p COM5 flash monitor
```

(use whatever port you found above instead of `COM5`)

`monitor` opens a live view of the board's serial log output (everything the firmware prints with `ESP_LOGI` / `ESP_LOGW` / `ESP_LOGE`) — this is your main window into what the board is actually doing. **To exit the monitor**, press `Ctrl+]`.

You can also do all of this from VS Code's **ESP-IDF** extension sidebar (the flame/chip icon) instead of typing commands — it has buttons for "Set Espressif Device Target", "Build", "Flash", and "Monitor".

Once this first flash succeeds, you're done with the BOOT+RESET dance. From here on, `idf.py flash` resets the board into bootloader mode itself over the USB connection — just plug in and flash normally (step 6 below). You only need to repeat the manual button sequence if a flash ever fails or hangs partway through (see Troubleshooting).

6. Make a change and reflash

Edit any file under `main/`, save it, then just run the same command again — no button-pressing needed:

```
idf.py -p COM5 flash monitor
```

Troubleshooting

"Failed to connect" / flashing times out / port not found - Redo the BOOT+RESET sequence from step 5, then retry the flash command immediately afterwards. - Make sure no other program (Arduino IDE, a serial monitor, a second VS Code window) has that same port open — only one program can hold a serial port at a time. - Try a different USB cable/port — not all USB-C cables carry data.

Build fails with missing component / CMake errors - Double check you picked **ESP-IDF v6.0** in the setup wizard — Command Palette → "ESP-IDF: Select ESP-IDF Version" if you need to switch. - Make sure you ran `idf.py set-target esp32s3` at least once before `idf.py build`. - Try a clean build: `idf.py fullclean` then `idf.py build`.

I want to see WiFi / the web dashboard from the full lab project - It's intentionally removed from this starter (see the comment block at the top of `main/main.c` for why — short version: transmitting without an antenna connected can damage the radio, and not every board here has one wired up). Ask your instructor if you need it for your specific board.

I have an OLED display wired up and want to use it - Open `main/main.c`, find `#define OLED_ENABLED 0` near the top, change the `0` to a `1`, save, and reflash.